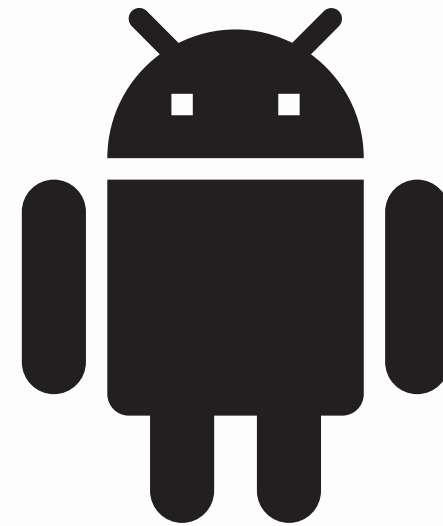




Combating Concept Drift with Explanatory Detection and Adaptation for Android Malware Classification

B123245017 B123245025

TABLE OF CONTENTS

01 實作目標

02 資料集

03 實作流程

04 實驗設定

05 實驗結果

06 討論與限制

01 實作目標

重製 DREAM 的核心概念，偵測 Android 惡意程式分類中的概念漂移，並在發現未知家族後進行模型適應。

實作範圍：

- 惡意程式家族分類器
- 對比式自編碼器偵測器
- 漂移分數計算
- Top-k 漂移樣本選取
- 模型適應與結果比較



02 資料集

由於原始論文使用的 Drebin 跟 MalRadar 資料集取得較不方便，因此我們改用較容易取得的 KronoDroid 資料集中的 emu_malware_v1.zip 進行重製實驗。

KronoDroid 提供 Android 應用程式的靜態特徵與惡意程式家族標籤，而我們從中選出樣本數最多的 8 個惡意程式家族作為實驗對象。

Drebin:

2010–2012 的 Android 惡意程式資料集
沒有完整的行為概念標籤，論文額外補上

MalRadar:

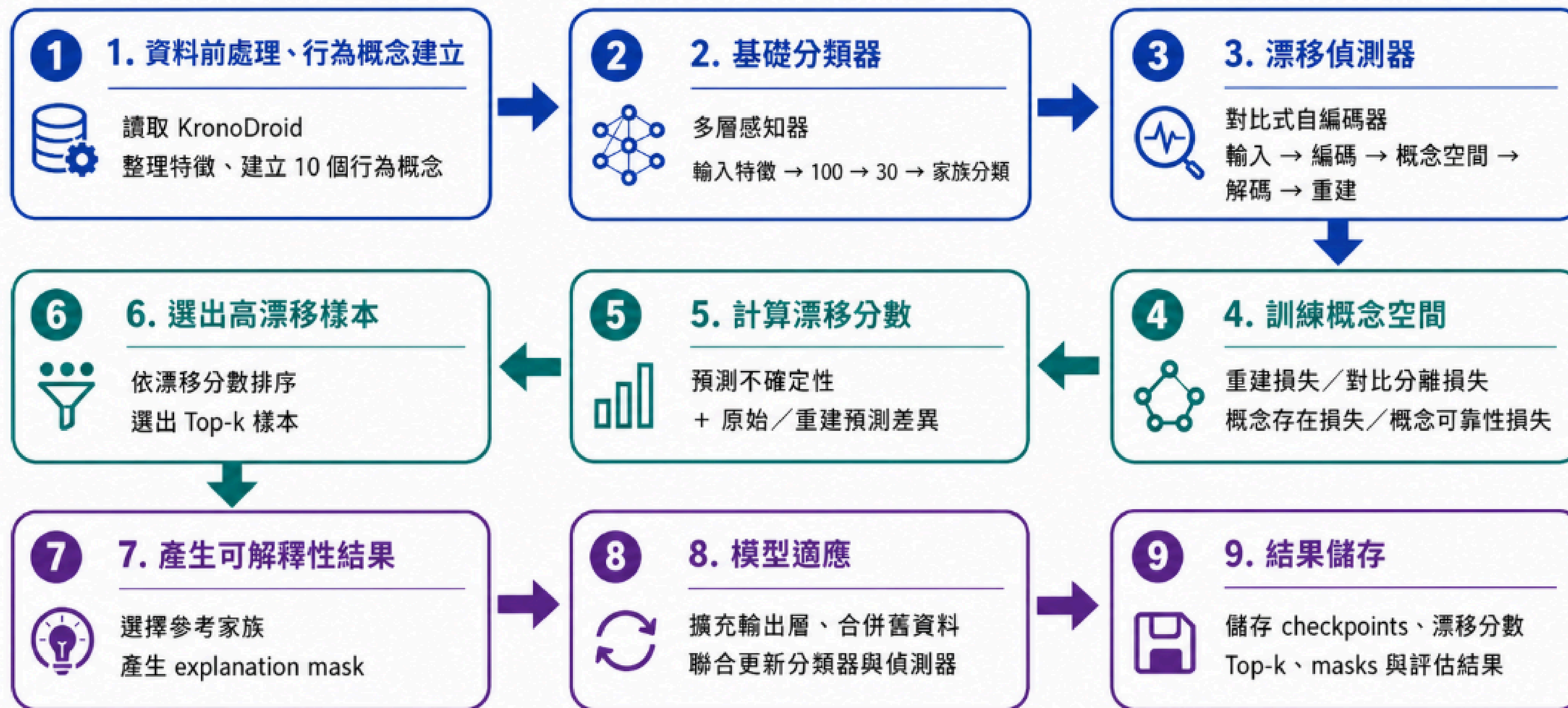
2015–2021 的 Android 惡意程式資料集
有定義惡意行為概念，作為概念空間中的行為概念來源

KronoDroid:

時間欄位大致涵蓋 2008–2020 年的 Android app 樣本
包含 benign apps 和 malware apps
沒有直接提供人工整理好的行為標籤

- Android app 樣本
- 惡意或正常標籤
- 惡意程式家族標籤
- 靜態特徵
- 動態特徵
- 時間欄位

03 實作流程



1. 資料前處理, 行為概念建立:

在進入模型訓練前, 程式會先將原始資料整理成分類器與
漂移偵測器可以使用的格式。

- 讀取 KronoDroid 資料
- 移除非特徵欄位
- 保留數值型特徵
- 進行特徵正規化
- 建立家族標籤

每個樣本會被轉換成特徵向量, 當分類器和偵測器的輸入。
程式也會建立 10 個行為概念標籤, 模擬顯式概念空間。

2.基礎分類器:

我們做了多層感知器分類器，作為原本已訓練好的惡意程式家族分類模型學習已知惡意程式家族的分類邏輯。

輸入特徵

- 隱藏層 100
- 隱藏層 30
- 家族分類結果

```
class DrebinMLP(nn.Module):  
    def __init__(self, input_dim, num_classes):  
        super().__init__()  
        self.feature = nn.Sequential(  
            nn.Linear(input_dim, 100),  
            nn.ReLU(),  
            nn.Linear(100, 30),  
            nn.ReLU(),  
        )  
        self.out = nn.Linear(30, num_classes)  
  
    def forward(self, x):  
        h = self.feature(x)  
        return self.out(h)
```

3. 漂移偵測器:

我們做了對比式自編碼器作為漂移偵測器，將輸入樣本編碼到概念空間，再重建回原始特徵空間。

輸入樣本

- 編碼器
- 概念空間
- 解碼器
- 重建樣本

```
self.encoder = nn.Sequential(  
    nn.Linear(input_dim, hidden_dim),  
    nn.ReLU(),  
    nn.Linear(hidden_dim, encoding_dim),  
    nn.ReLU(),  
)  
  
self.decoder = nn.Sequential(  
    nn.Linear(encoding_dim, hidden_dim),  
    nn.ReLU(),  
    nn.Linear(hidden_dim, input_dim),  
    nn.Sigmoid(),  
)  
  
def forward(self, x):  
    z = self.encoder(x)  
    x_hat = self.decoder(z)  
    concept_probs = self.concept_head(z[:, :self.num_concepts])  
    return x_hat, z, concept_probs
```

4.訓練概念空間:

參照論文，我們使用損失函數讓概念空間更穩定。

- 重建損失：讓重建樣本接近原始樣本
- 對比分離損失：讓同一家族樣本在概念空間中靠近，不同家族樣本分開
- 概念存在損失：讓顯式概念維度對應到行為概念標籤
- 概念可靠性損失：讓原始樣本與重建樣本通過分類器後的預測保持一致

```
loss = (  
    lambda_rec * loss_rec  
    + lambda_sep * loss_sep  
    + lambda_rel * loss_rel  
    + lambda_pre * loss_pre  
)
```

5.計算漂移分數:

每個新樣本都會計算漂移分數。

漂移分數 = 分類器對樣本的預測不確定性 + 原始樣本與重建樣本的預測差異

漂移分數越高，代表這個樣本對目前分類器來說越不可靠，也越可能是未知家族或新的行為分布。

```
logits = classifier(xb)
pseudo_ce = pseudo_cross_entropy_per_sample(logits)
x_hat, _, _ = detector(xb)
rel = concept_reliability_loss_per_sample(classifier, xb, x_hat)
score = pseudo_ce + lambda_rel * rel
```

6.選出高漂移樣本:

計算完所有樣本的漂移分數後，我們依照分數由高到低排序，選出前k個最可疑的樣本。

這些樣本會作為後續模型適應的輸入，由下一部分負責使用這些結果進行模型更新。

```
def select_top_k_by_score(scores, k):  
    k = min(k, len(scores))  
    return np.argsort(scores)[::-1][:k]
```

7.產生可解釋性結果:

這個部分會找出哪些概念維度對漂移判斷最重要，並產生 explanation mask，幫助我們理解模型認為樣本偏離已知家族的原因。

漂移樣本

- 選擇參考家族
- 比較概念空間差異
- 產生 explanation mask

```

def compute_class_centroids(detector, X_train, y_train, batch_size=32):
    detector.eval()
    X_tensor = torch.tensor(X_train, dtype=torch.float32)
    dl = DataLoader(TensorDataset(X_tensor), batch_size=batch_size, shuffle=False)
    z_all = []
    with torch.no_grad():
        for (xb,) in dl:
            xb = xb.to(device)
            _, z, _ = detector(xb)
            z_all.append(z.cpu())

    z_all = torch.cat(z_all, dim=0)
    y_tensor = torch.tensor(y_train, dtype=torch.long)
    centroids = {}
    closest_samples = {}
    for cls in sorted(np.unique(y_train)):
        idx = torch.where(y_tensor == int(cls))[0]
        z_cls = z_all[idx]
        centroid = z_cls.mean(dim=0)
        dist = torch.sqrt(torch.sum((z_cls - centroid) ** 2, dim=1) + 1e-10)
        nearest_local = torch.argmin(dist).item()
        nearest_global = idx[nearest_local].item()
        centroids[int(cls)] = centroid.to(device)
        closest_samples[int(cls)] = X_train[nearest_global]
    return centroids, closest_samples

```

計算每個已知家族的概念空間中心

找出最接近這個概念中心的訓練樣本，作為該家族的參考樣本，並比較概念空間差異

```
with torch.no_grad():
    pred_label = classifier(x_drift).argmax(dim=1).item()
    x_ref_np = closest_samples[pred_label]
    x_ref = torch.tensor(x_ref_np[None, :], dtype=torch.float32, device=device)
    _, z_drift, _ = detector(x_drift)
    _, z_ref, _ = detector(x_ref)
    c_y = centroids[pred_label].detach()
    ref_pred = F.softmax(classifier(x_ref), dim=1).detach()
```

最佳化mask後產生最終的 explanation mask

```
distance_loss = torch.sqrt(torch.sum((z_prime_check - c_y) ** 2, dim=1) + 1e-10).mean()
logits_prime = classifier(x_prime)
pseudo_ce = F.cross_entropy(
    logits_prime,
    logits_prime.argmax(dim=1),
    reduction="mean",
)
log_probs_prime = F.log_softmax(logits_prime, dim=1)
rel_loss = -(ref_pred.repeat(batch_size, 1) * log_probs_prime).sum(dim=1).mean()
uncertainty_loss = pseudo_ce + rel_loss
regularization = torch.sum(torch.abs(m)) / batch_size
l2_reg = torch.sqrt(torch.sum(m * m) + 1e-10) / batch_size
loss = distance_loss + alpha_u * uncertainty_loss + active_lambda * (regularization + l2_reg)
```

```
if best_mask is None:
    best_mask = torch.sigmoid(p.detach())
binary_mask = (best_mask > 0.5).float().cpu().numpy().astype(np.int64)
return binary_mask
```

8. 模型適應:

當 DREAM 偵測到高漂移樣本後:

1. 將 Top-k 漂移樣本加入訓練資料
2. 為未知家族新增一個輸出類別
3. 擴充分類器輸出層
4. 聯合更新 Classifier 與 Detector
5. 使用 Dynamic LR Schedule 穩定 Detector

```
classifier = expand_krono_classifier(  
    old_classifier, num_total_classes  
).to(device)
```

```
detector = copy.deepcopy(old_detector).to(device)
```

```
X_selected = X_test[selected_idx]  
y_selected = y_test_for_adapt[selected_idx]  
c_selected = concept_test[selected_idx]  
m_selected = mask_test[selected_idx]
```

- Expand classifier output layer
- Select top-k drift samples

```
X_adapt = np.concatenate([X_train, X_selected], axis=0)  
y_adapt = np.concatenate([y_train, y_selected], axis=0)  
c_adapt = np.concatenate([concept_train, c_selected], axis=0)  
m_adapt = np.concatenate([mask_train, m_selected], axis=0)
```

- Add selected drift samples into adaptation set
- Use labels + behavior concepts

```
cls_loss = F.cross_entropy(logits, yb)

rec_loss = reconstruction_loss(x_hat, xb)
sep_loss = cade_contrastive_separation_loss(z, yb)
rel_loss = concept_reliability_loss(classifier, xb, x_hat)
pre_loss = concept_presence_loss(concept_probs, cb, mb)

total = (
    cls_loss
    + lambda_rec * rec_loss
    + lambda_sep * sep_loss
    + lambda_rel * rel_loss
    + lambda_pre * pre_loss
)
```

Update classifier and detector together

9.結果儲存:

在模型訓練、漂移偵測、解釋產生與模型適應完成後，程式會將重要結果儲存下來，方便後續分析與比較。

- 原始分類器權重
- 原始漂移偵測器權重
- 適應後分類器權重
- 適應後漂移偵測器權重
- 漂移分數
- Top-k 被選出的樣本索引
- Explanation masks
- 實驗評估結果

04 實驗設定

從 KronoDroid 中選出樣本數最多的 8 個惡意程式家族

- Airpush/StopSMS (Total samples: 20,774)
- Boxer
- Malap
- FakeInst
- Agent
- Locker/SLocker Ransomware
- BankBot
- FakeApp

Behavioral Concepts Used in Reproduction

10個對應論文中的顯式概念空間的行為概念：

- 隱私竊取
- 簡訊與通話濫用
- 遠端控制
- 金融竊取
- 勒索行為
- 無障礙服務濫用
- 權限提升
- 隱蔽下載
- 侵入式廣告
- 付費服務濫用

它們是根據 KronoDroid 的權限與靜態特徵推導出來的，不是原始論文的人工標註。

Hold-out Family & Simulated Human Feedback

- 我們使用了 hold-out family 的方式模擬未知家族造成的概念漂移。
- 訓練階段中先移除了某一個惡意程式家族，讓模型只學習其他已知家族，測試階段再把被移除的家族加回測試資料中，觀察模型是否能偵測出這個未知家族。
- 因為我們缺乏惡意程式分析的知識，所以專家參與標註及回饋流程的部分，我們使用資料集中的真實標籤來模擬專家標註。

Setting: Hold-out Family & Budget Test

Hold-out Families

- BankBot
- FakeApp
- Boxer

Top-k Budget

- $k = 20$
- $k = 50$
- $k = 100$

模擬不同人工分析成本

Training Configuration

Classifier epochs = 30
Detector epochs = 30
Adaptation epochs = 30

Batch size = 32
Learning rate = $4e-4$
Encoding dim = 32
Hidden dim = 512

05 實驗結果

All Hold-out Results

holdout_family	budget	auc	true_drift	before_all_accuracy	before_all_f1_macro	after_accuracy	after_f1_macro
BankBot	20	0.86	4	0.71	0.66	0.72	0.67
BankBot	50	0.86	13	0.71	0.66	0.74	0.7
BankBot	100	0.86	51	0.71	0.66	0.74	0.71
FakeApp	20	0.84	20	0.74	0.7	0.76	0.72
FakeApp	50	0.84	46	0.74	0.7	0.76	0.73
FakeApp	100	0.84	85	0.74	0.7	0.77	0.74
Boxer	20	0.78	1	0.45	0.68	0.46	0.7
Boxer	50	0.78	9	0.45	0.68	0.46	0.7
Boxer	100	0.78	59	0.45	0.68	0.55	0.74

1. Drift Detection Performance

Holdout	AUC
BankBot	0.862
FakeApp	0.845
Boxer	0.781

- DREAM-style detector 在三個 hold-out family 上都能成功辨識 concept drift。
- BankBot 與 FakeApp 的 AUC 超過 0.84。
- Boxer 相對困難，但仍有 0.78。

2. Budget Analysis

holdout_family	budget	true_drift
BankBot	20	4
BankBot	50	13
BankBot	100	51
FakeApp	20	20
FakeApp	50	46
FakeApp	100	85

- 當 budget 增加時，能選到更多真正的 drift samples。
- 因此 adaptation 擁有更多有價值的資訊。

3. Adaptation Performance

Holdout	Before F1	After F1
BankBot	0.659	0.706
FakeApp	0.699	0.740
Boxer	0.677	0.740

- 三個 hold-out family 都出現改善。
- 代表 adaptation 確實讓模型學會新的家族資訊。
- 其中 Boxer 提升最大。

4. Explanation Example

- Explicit concept masks first sample: [0 0 **1 1 1 1 1 1 1** 0]
- Explicit concepts: ['privacy_stealing', 'sms_call_abuse', '**remote_control**', '**bank_financial_stealing**', '**ransom**', '**accessibility_abuse**', '**privilege_escalation**', '**stealthy_download**', '**aggressive_advertising**', 'premium_service']
- 甚至有 [1 1 1 1 1 1 1 1 1 1]

06 討論與限制

What worked

- Hold-out drift successfully detected
- AUC 0.78~0.86
- Adaptation improves macro-F1
- More budget generally improves performance

Limitations

- Concept labels are heuristically generated
- Explainer masks too dense (mask 幾乎全 1)
- No baseline comparison (CADE, Active L)

Conclusion

- Successfully reproduced the main DREAM workflow, including drift detection, concept-space explanation, and adaptation.
- The detector **achieved AUC up to 0.862** and adaptation **consistently improved macro-F1** across all tested hold-out families.
- However, explanation masks remained overly dense, suggesting that reproducing high-quality concept-level explanations remains challenging without analyst-defined behavioral concepts.

THANK YOU